

Tarea 5: Configuración cliente-servidor con LAMP

Esta tarea se compone de **3 partes** y cada una de ellas con un entregable asociado.

La tarea se debe de entregar mediante un **fichero .zip**

(*apellido1_apellido2_nombre_Tarea5.zip*) que contenga los 3 entregables.

Parte 1 - Instalación LAMP en máquina virtual

Entregable

- Documento en pdf con las capturas que evidencien la correcta instalación (*Parte1.pdf*)
 - Captura de la configuración de la máquina virtual
 - Captura de la versión de Apache instalada y de la página de apache abierta en Firefox (<http://localhost>)
 - Captura de MariaDB instalada (comando *mysql --version*)
 - Captura de instalación PHP (comando *php -v*)
 - Captura de conexión de PHP con servidor web
 - Captura de conexión de PHP con base de datos
 - Captura de phpMyAdmin funcionando desde el navegador

Desarrollo

1. Instalación de la máquina con Ubuntu 22.04.5 LTS

En esta primera parte vas a instalar un **entorno LAMP** (Linux + Apache + MariaDB/MySQL + PHP) sobre una **máquina virtual** (en VirtualBox) **con Ubuntu** y elaborarás una **guía documentando el proceso**.

Si aún no tienes **VirtualBox** descargado, realiza la descarga desde el [siguiente link](#).

Una vez configurado **descarga la ISO de [Ubuntu Desktop](#) versión 22.04.5 LTS**:

ubuntu® releases

Ubuntu 22.04.5 LTS (Jammy Jellyfish)

Select an image

Ubuntu is distributed on three types of images described below.

Desktop image

The desktop image allows you to try Ubuntu without changing your computer at all, and at your option to install it permanently later. This type of image is what most people will want to use. You will need at least 1024MiB of RAM to install from this image.

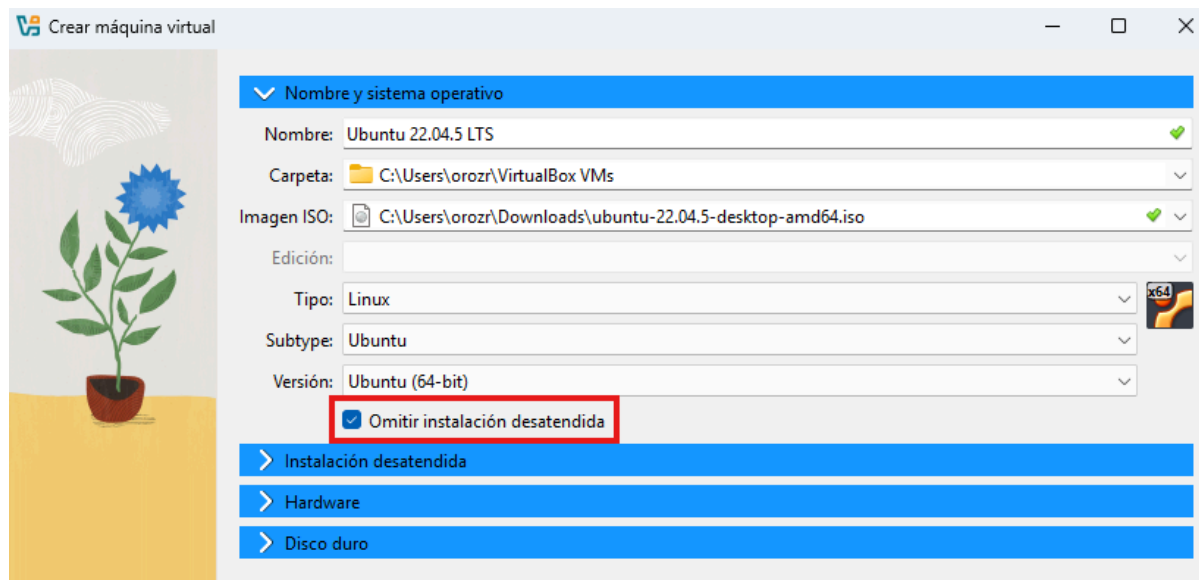
64-bit PC (AMD64) desktop image

Choose this if you have a computer based on the AMD64 or EM64T architecture (e.g., Athlon64, Opteron, EM64T Xeon, Core 2). Choose this if you are at all unsure.

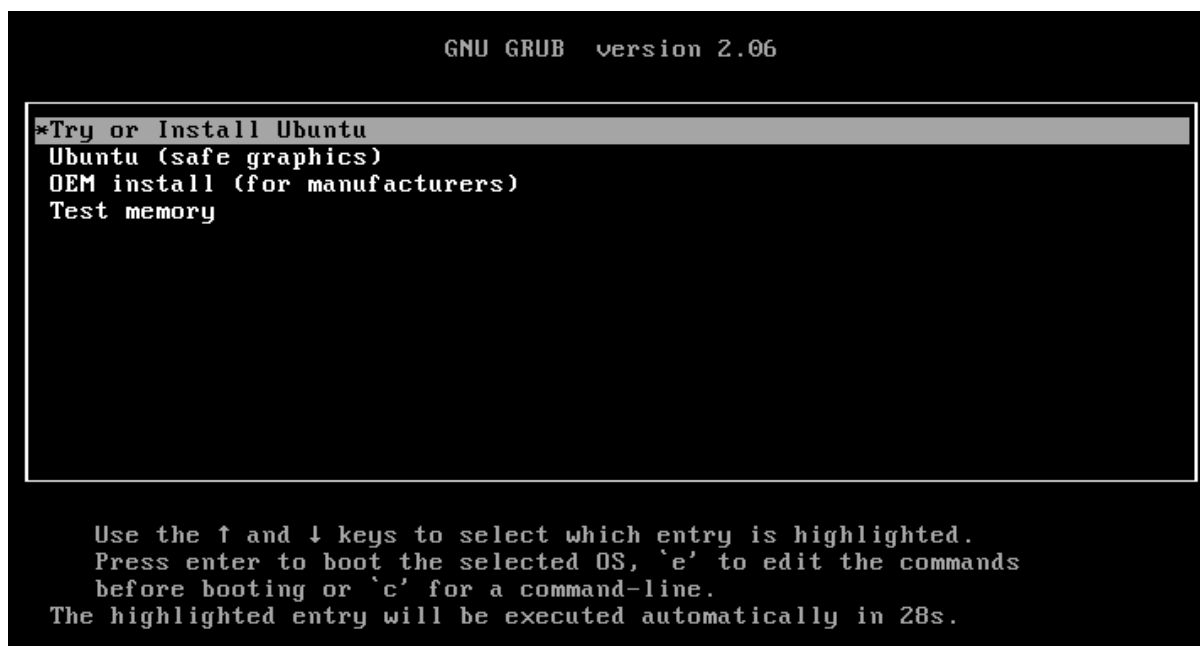
Crea una máquina virtual (MV) en VirtualBox utilizando la ISO descargada con las siguientes características:

- CPU: 2 cores
- RAM: 4GB
- Disco: 25GB

Selecciona “Omitir instalación desatendida”:

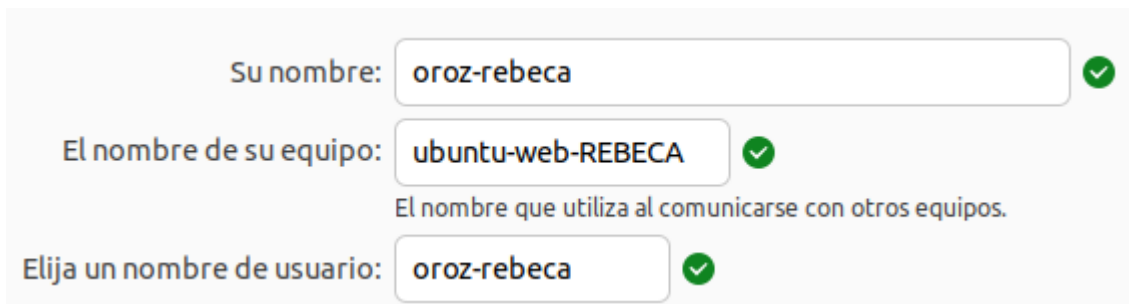


Selecciona esta opción:

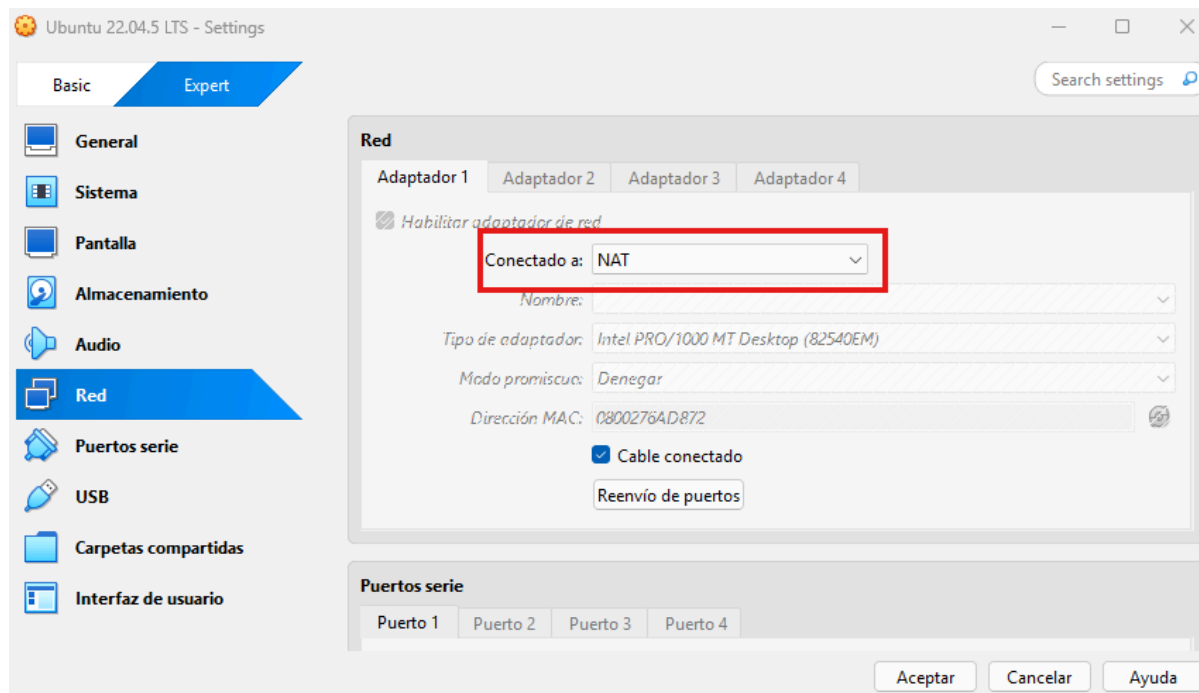


Realiza el proceso de instalación os pedirá los siguientes campos y debéis de incluir la siguiente información:

- Nombre: **apellido-nombre**
- Nombre de la máquina virtual: **ubuntu-web-NOMBRE**



Tras terminar la instalación, revisa que la **configuración de red** sea a través de NAT (para que nuestra máquina pueda salir a Internet a través de nuestro ordenador). Si no lo está, debemos de configurarlo como NAT:



Ahora sí, volvemos a encender la MV y **actualizamos los paquetes** a través del terminal:

- `sudo apt update`
- `sudo apt upgrade`

Una vez instalados los paquetes reiniciamos la máquina (`sudo reboot`). En este punto tendremos ya la máquina lista, por lo que instalamos LAMP.

2. Instalación LAMP

LAMP es un conjunto de aplicaciones de software y de tecnologías (**L**inux **A**pache **M**ySQL/**M**ariaDB **P**HP), por lo que iremos instalando cada una de ellas:

2.1. Apache

Desde la terminal instala Apache desde el rol de administrador:

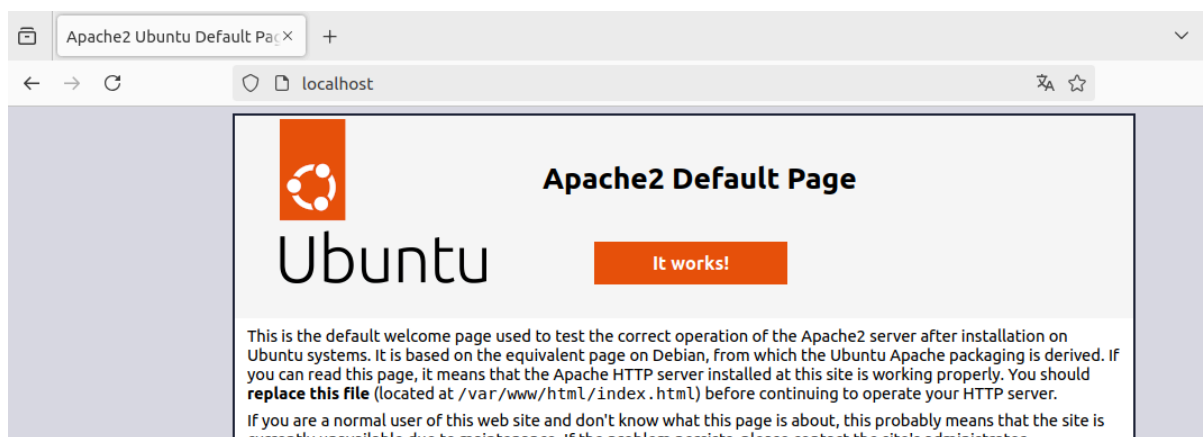
- `sudo apt install apache2`

Revisamos si se ha instalado correctamente (captura a incluir en la tarea):

- `apache2 -v`

```
oroz-rebeca@ubuntu-web-REBECA:~$ apache2 -v
Server version: Apache/2.4.52 (Ubuntu)
Server built: 2025-08-11T12:10:10
```

Lo comprobamos también desde el navegador:



2.2. MariaDB

Usaremos MariaDB como motor de bases de datos. MariaDB es un fork de MySQL totalmente compatible, creado por los desarrolladores originales de MySQL.

Desde la terminal instala MariaDB desde el rol de administrador:

- `sudo apt install mariadb-server`

Revisamos si se ha instalado correctamente (captura a incluir en la tarea):

- `mariadb - --version`

```
oroz-rebeca@ubuntu-web-REBECA:~$ mariadb --version
mariadb Ver 15.1 Distrib 10.6.22-MariaDB, for debian-linux-gnu (x86_64) using
EditLine wrapper
```

2.3. PHP

PHP actúa como lenguaje de programación del lado del servidor y requiere extensiones adicionales para integrarse con Apache y comunicarse con MariaDB. Sin estas extensiones, Apache no podría procesar archivos .php ni PHP podría acceder a la base de datos.

- `sudo apt install php`
- `sudo apt install libapache2-mod-php`
- `sudo apt install php-mysql`
- `sudo systemctl restart apache2`

Comprobamos si se ha instalado correctamente:

- `sudo php -v`

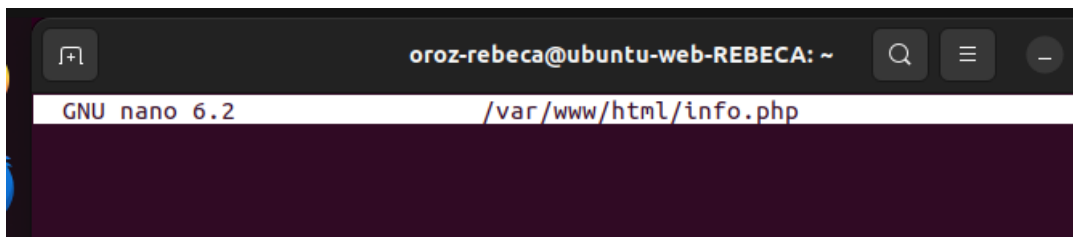
```
oroz-rebeca@ubuntu-web-REBECA:~$ php -v
PHP 8.1.2-1ubuntu2.23 (cli) (built: Jan 7 2026 08:37:41) (NTS)
Copyright (c) The PHP Group
Zend Engine v4.1.2, Copyright (c) Zend Technologies
with Zend OPcache v8.1.2-1ubuntu2.23, Copyright (c), by Zend Technologies
```

Una vez instalado, realizamos dos pruebas finales: Prueba de conexión de PHP con servidor web y la prueba de conexión de PHP con base de datos:

- **Prueba de conexión de PHP con el servidor web**

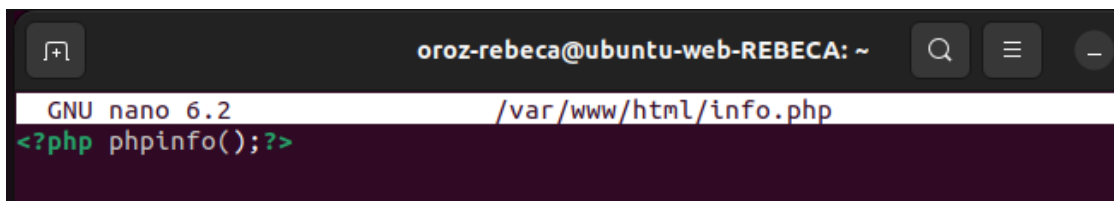
Creamos un archivo de prueba:

- `sudo nano /var/www/html/info.php`



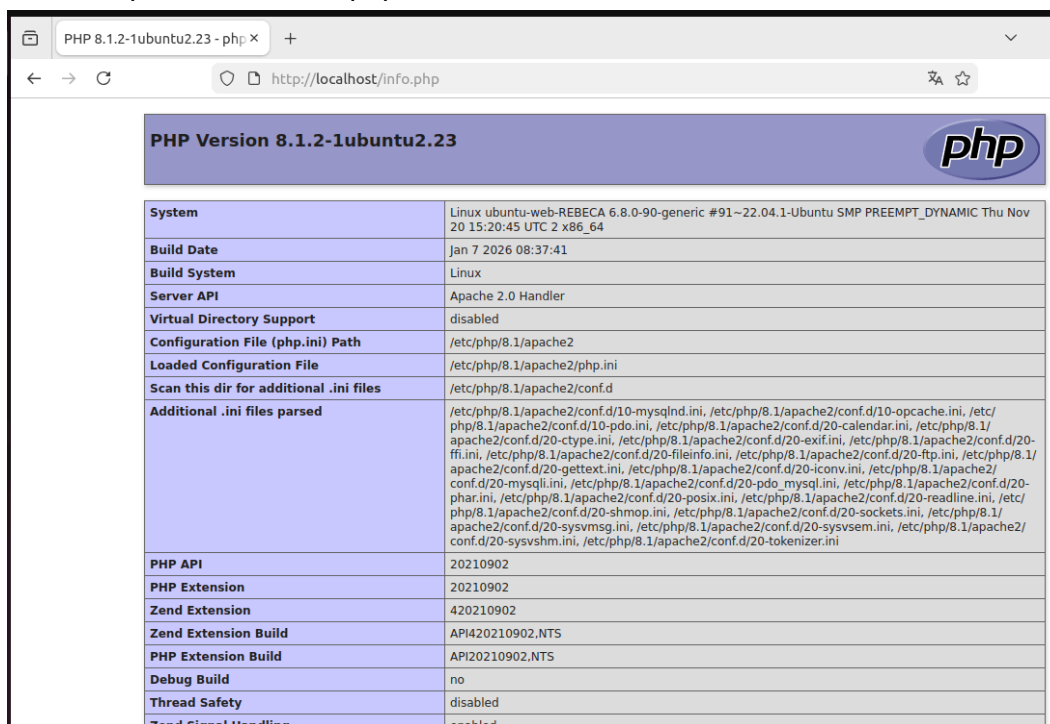
Añadimos el siguiente contenido:

- `<?php phpinfo(); ?>`



Lo comprobamos accediendo desde el navegador del sistema:

- `http://localhost/info.php`



PHP Version 8.1.2-1ubuntu2.23	
System	Linux ubuntu-web-REBECA 6.8.0-90-generic #91~22.04.1-Ubuntu SMP PREEMPT_DYNAMIC Thu Nov 20 15:20:45 UTC 2 x86_64
Build Date	Jan 7 2026 08:37:41
Build System	Linux
Server API	Apache 2.0 Handler
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc/php/8.1/apache2
Loaded Configuration File	/etc/php/8.1/apache2/php.ini
Scan this dir for additional .ini files	/etc/php/8.1/apache2/conf.d
Additional .ini files parsed	/etc/php/8.1/apache2/conf.d/10-mysqld.ini, /etc/php/8.1/apache2/conf.d/10-opcache.ini, /etc/php/8.1/apache2/conf.d/10-pdo.ini, /etc/php/8.1/apache2/conf.d/20-calendar.ini, /etc/php/8.1/apache2/conf.d/20-ctype.ini, /etc/php/8.1/apache2/conf.d/20-exif.ini, /etc/php/8.1/apache2/conf.d/20-ffi.ini, /etc/php/8.1/apache2/conf.d/20-fileinfo.ini, /etc/php/8.1/apache2/conf.d/20-ftp.ini, /etc/php/8.1/apache2/conf.d/20-gettext.ini, /etc/php/8.1/apache2/conf.d/20-iconv.ini, /etc/php/8.1/apache2/conf.d/20-mysqli.ini, /etc/php/8.1/apache2/conf.d/20-pdo_mysql.ini, /etc/php/8.1/apache2/conf.d/20-phar.ini, /etc/php/8.1/apache2/conf.d/20-posix.ini, /etc/php/8.1/apache2/conf.d/20-readline.ini, /etc/php/8.1/apache2/conf.d/20-shmop.ini, /etc/php/8.1/apache2/conf.d/20-sockets.ini, /etc/php/8.1/apache2/conf.d/20-sysvmsg.ini, /etc/php/8.1/apache2/conf.d/20-sysvsem.ini, /etc/php/8.1/apache2/conf.d/20-sysvshm.ini, /etc/php/8.1/apache2/conf.d/20-tokenizer.ini
PHP API	20210902
PHP Extension	20210902
Zend Extension	420210902
Zend Extension Build	API420210902.NTS
PHP Extension Build	API20210902.NTS
Debug Build	no
Thread Safety	disabled
Zend Signal Handling	enabled

- **Prueba de conexión de PHP con la base de datos**

a) Entramos a MariaDB y creamos una base de datos y un usuario

- `sudo mariadb`

```
oroz-rebeca@ubuntu-web-REBECA:~$ sudo mariadb
[sudo] contraseña para oroz-rebeca:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 32
Server version: 10.6.22-MariaDB-0ubuntu0.22.04.1 Ubuntu 22.04

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

Creamos una base de datos (testdb):

```
MariaDB [(none)]> CREATE DATABASE testdb;
Query OK, 1 row affected (0,011 sec)
```

Creamos un usuario (test) y contraseña (123) y le asignamos la base de datos:

```
MariaDB [(none)]> CREATE USER 'test'@'localhost' IDENTIFIED BY '123';
Query OK, 0 rows affected (0,072 sec)
```

```
MariaDB [(none)]> GRANT ALL PRIVILEGES ON testdb.* TO 'test'@'localhost';
Query OK, 0 rows affected (0,011 sec)
```

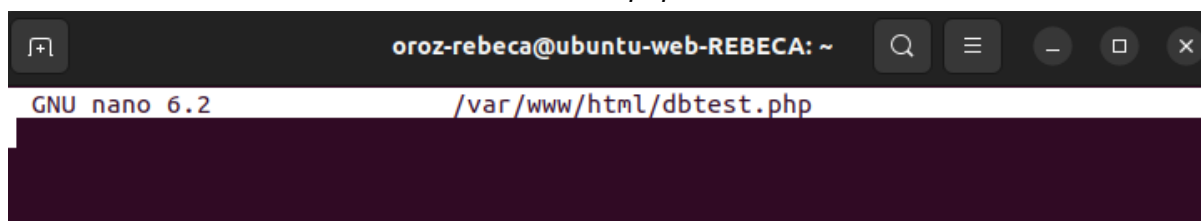
Actualizamos los privilegios y salimos:

```
MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0,019 sec)

MariaDB [(none)]> EXIT
```

b) Creamos un archivo de conexión

- `sudo nano /var/www/html/dbtest.php`



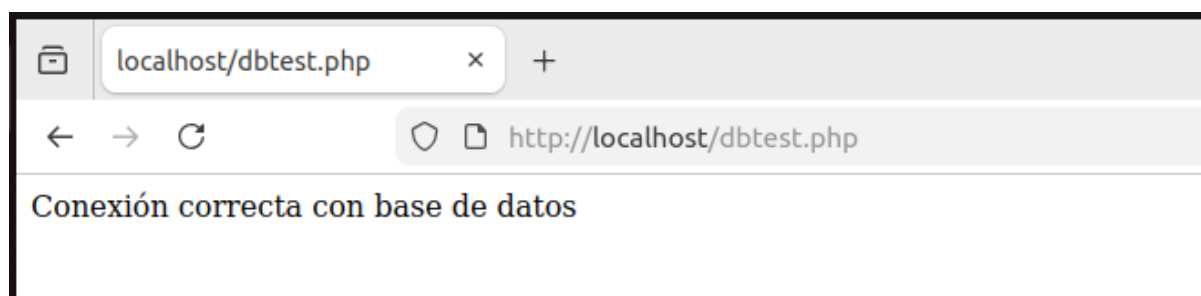
Añadimos el siguiente contenido:



```
oro-rebeca@ubuntu-web-REBECA: ~  
GNU nano 6.2 /var/www/html/dbtest.php  
<?php  
$conexion = new mysqli("localhost","test","123","testdb");  
  
if ($conexion->connect_error) {  
    echo "Error de conexión";  
} else { echo "Conexión correcta con base de datos";}  
  
?>
```

Lo comprobamos accediendo desde el navegador del sistema:

- `http://localhost/dbtest.php`



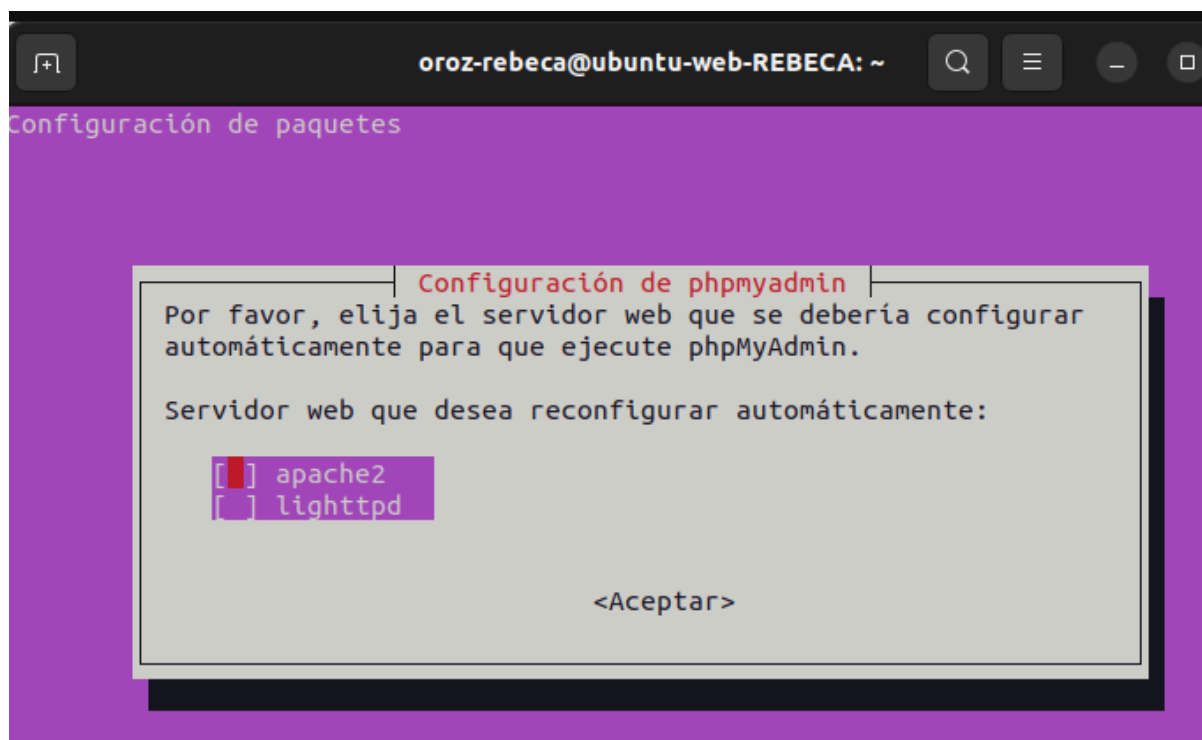
3. Instalación de phpMyAdmin

Aunque la conexión entre PHP y MariaDB ya funciona correctamente, vamos a instalar **phpMyAdmin** como herramienta visual que facilite la gestión de la base de datos.

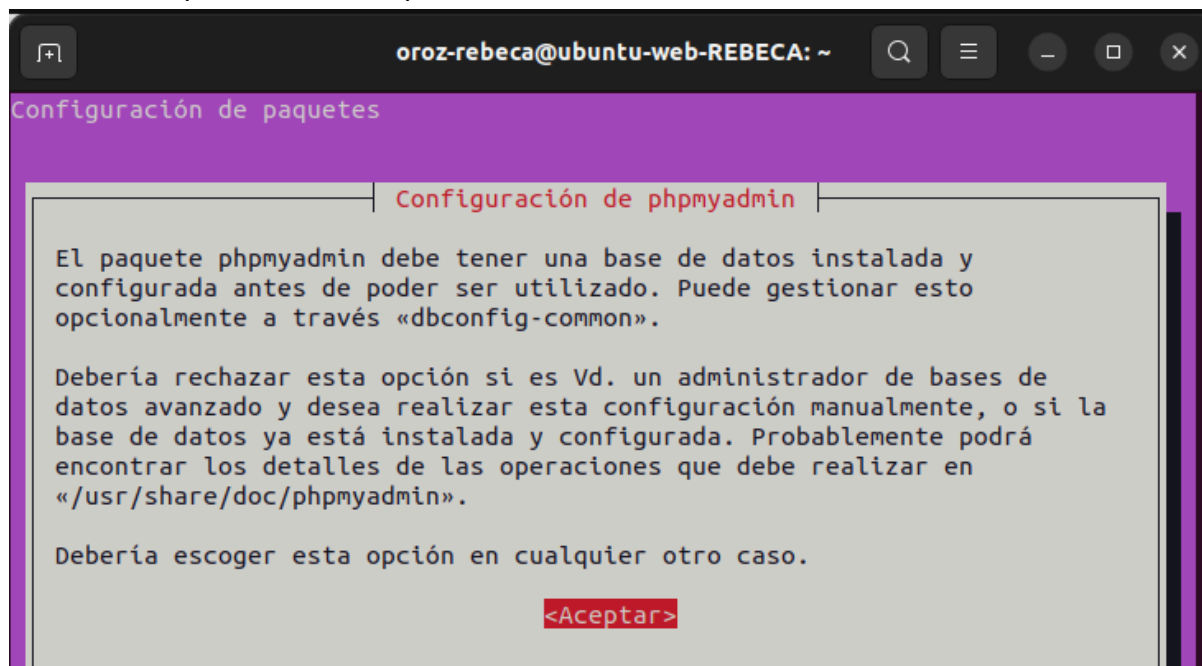
Desde la terminal instala phpmyadmin:

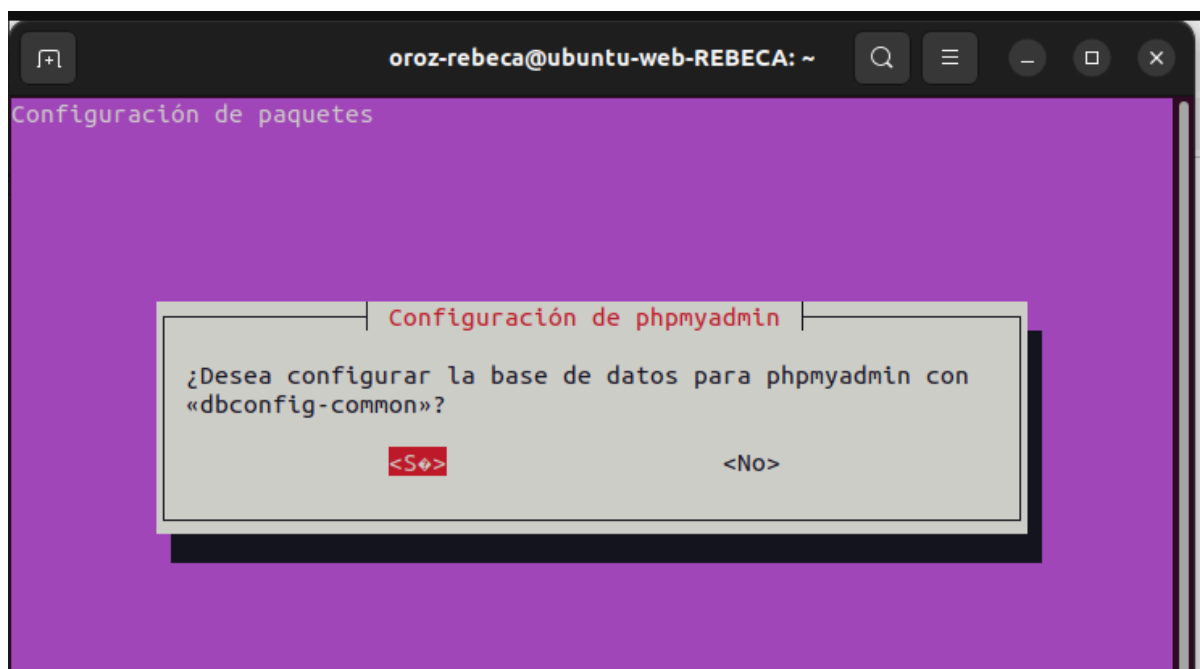
- `sudo apt install phpmyadmin`

Durante la instalación nos pedirá que seleccionemos el servidor web, seleccionamos `apache2`:

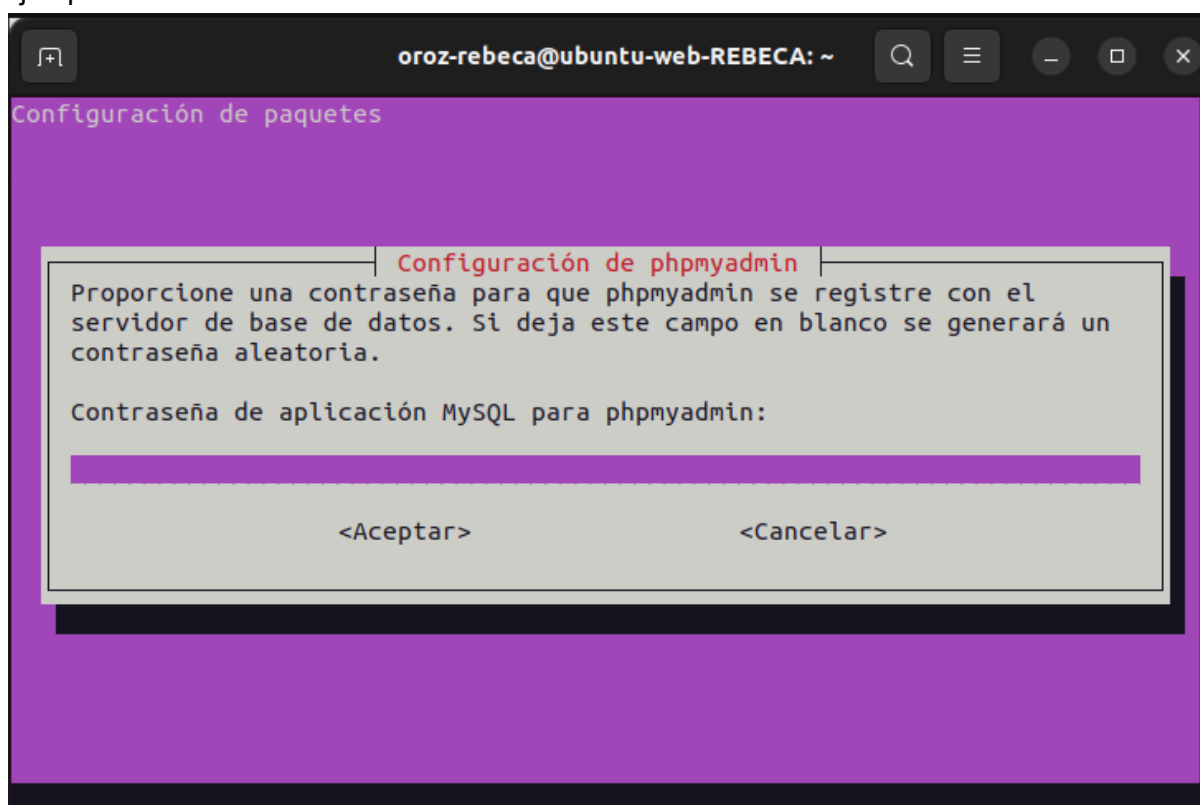


Se detecta que ya tenemos MariaDB instalado y nos pregunta si queremos que configure la base interna que necesita, aceptamos:





Incluimos una contraseña que va a usar phpMyAdmin para registrarse en MariaDB, por ejemplo *admin123*:



En Ubuntu la instalación de phpMyAdmin no crea automáticamente el acceso desde el navegador. Para habilitarlo creamos un enlace simbólico (atajo) dentro del servidor web:

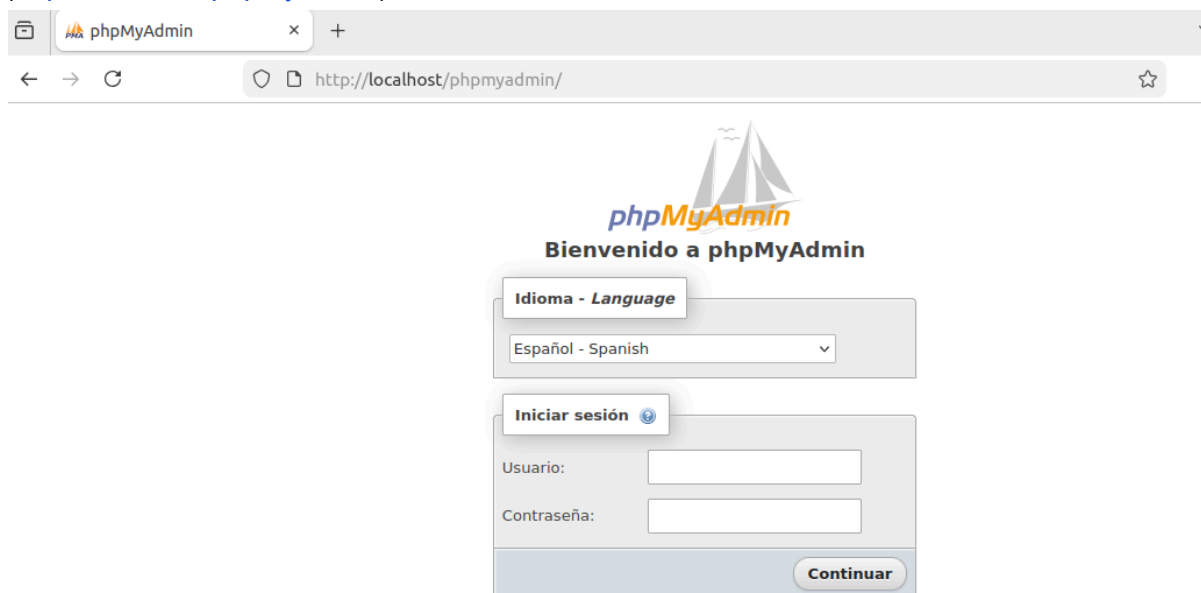
- `sudo ln -s /usr/share/phpmyadmin /var/www/html/phpmyadmin`

```
oroz-rebeca@ubuntu-web-REBECA:~$ sudo ln -s /usr/share/phpmyadmin /var/www/html/phpmyadmin
```

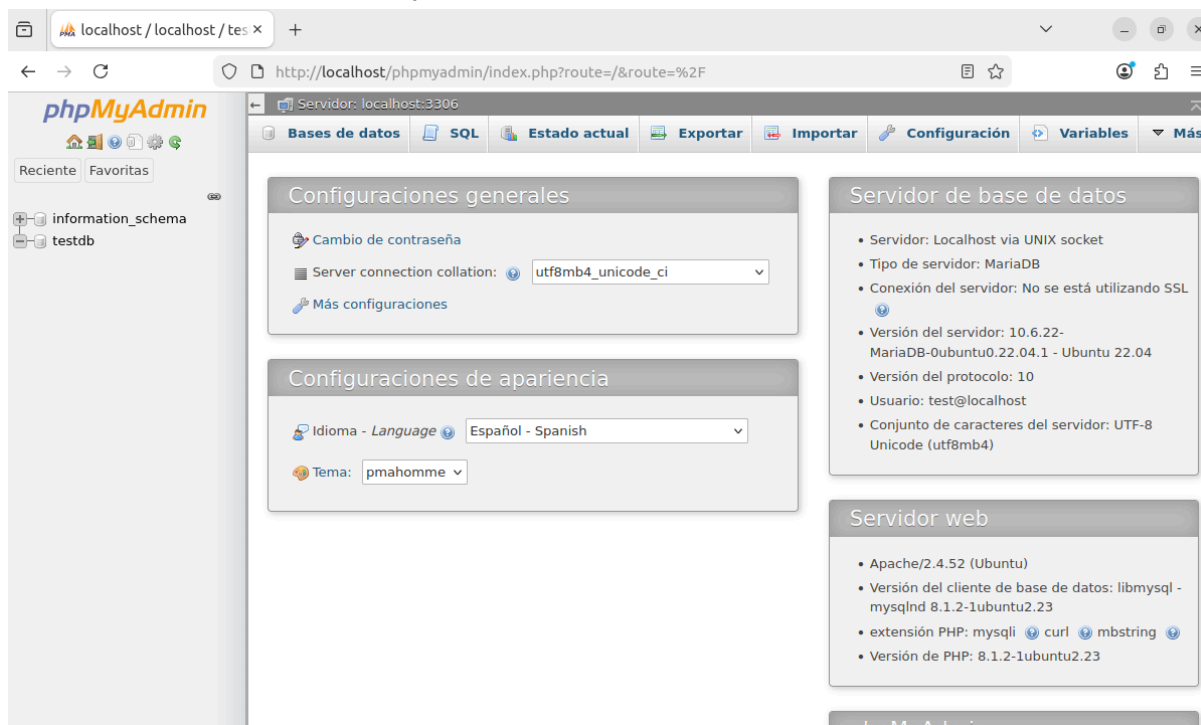
Reiniciamos Apache para aplicar cambios:

- `sudo systemctl restart apache2`

Comprobamos que funciona correctamente conectándonos desde el navegador (<http://localhost/phpmyadmin/>):



Podemos entrar con el usuario y la contraseña que habíamos creado por comandos:



Por último, para desarrollar el código de la parte web (HTML, JS, PHP) instalamos [Visual Studio Code](#) en la máquina virtual desde el navegador:



Instalamos la aplicación:



Parte 2 - Primer cliente-servidor

Entregable

- Documento (*Parte2.pdf*) **con explicaciones** y las siguientes captura:
 - Captura de la base de datos y tabla creadas en phpMyAdmin
 - Captura del formulario funcionando en el navegador
 - Captura del mensaje o resultado tras el envío del formulario
 - Captura de phpMyAdmin mostrando los registros insertados
 - Archivos utilizados en la práctica (.html y .php)

Desarrollo

En esta parte vamos a desarrollar una pequeña aplicación web que nos permita comprender el flujo básico cliente-servidor y la interacción con una base de datos.

Nota: para el correcto funcionamiento los archivos *web.html* e *insertar.php* deben de ubicarse en el siguiente directorio: */var/www/html*

Como esta carpeta pertenece a root, no podrás trabajar directamente sobre ella. Para facilitar la gestión te recomiendo que des permisos a tu usuario para que puedas trabajar directamente:

- `sudo chown -R $USER:$USER /var/www/html`
- `sudo chmod -R 755 /var/www/html`

```
oroz-rebeca@ubuntu-web-REBECA:~$ sudo chown -R oroz-rebeca:oroz-rebeca /var/www/html
oroz-rebeca@ubuntu-web-REBECA:~$ sudo chmod -R 755 /var/www/html
```

Con estos comandos asignamos el directorio */var/www/html* al usuario *oroz-rebeca*, y configuramos los permisos para que el usuario pueda editar los archivos y el servidor web pueda leerlos.

1. Crea una base de datos y una tabla

Desde phpMyAdmin, crear una base de datos llamada ***usuariosdb*** y una tabla llamada ***usuarios***, que contenga al menos los siguientes campos:

- id (clave primaria, incremento automático)
- nombre
- email

Nota: el usuario test no posee privilegios para crear bases de datos. Para ello podemos mediante consola modificar la contraseña del usuario root a "admin123":

```
oroz-rebeca@ubuntu-web-REBECA:~$ sudo mariadb
[sudo] contraseña para oroz-rebeca:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 108
Server version: 10.6.22-MariaDB-0ubuntu0.22.04.1 Ubuntu 22.04
```

```
MariaDB [(none)]> ALTER USER 'root'@'localhost' IDENTIFIED BY 'admin123';
Query OK, 0 rows affected (0,028 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0,034 sec)
```

2. Crea un formulario web (cliente)

Crea un formulario simple en un archivo HTML (web.html) con Visual Studio Code que permita al usuario introducir un nombre y un email y enviar los datos al servidor mediante el método POST.

El formulario enviará los datos al archivo insertar.php, que se encargará de guardarlos en la base de datos.

3. Procesa los datos en el servidor

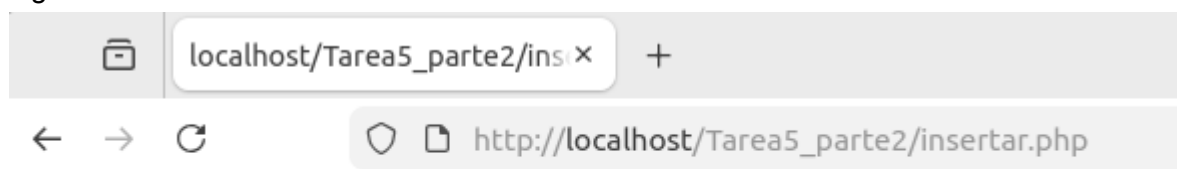
Crea un archivo PHP (insertar.php) que reciba los datos enviados desde el formulario y los inserte en la tabla creada en la base de datos. Como el trabajo con bases de datos lo veremos en la UT8 en profundidad, dejo aquí el código mínimo para su funcionamiento:

```
insertar.php
1  <?php
2
3  //Recogemos los datos enviados desde el formulario
4  $nombre=$_POST['nombre'];
5  $email=$_POST['email'];
6
7  //Conectamos con la base de datos
8  $conexion = new mysqli("localhost","root","admin123","usuariosdb");
9
10 // Insertamos los datos en la tabla usuarios
11 $conexion->query("INSERT INTO usuarios (nombre, email) VALUES('$nombre','$email')");
12
13 //Mostramos un mensaje de confirmación al usuario
14 echo "Datos guardados correctamente";
15
16 ?>
```

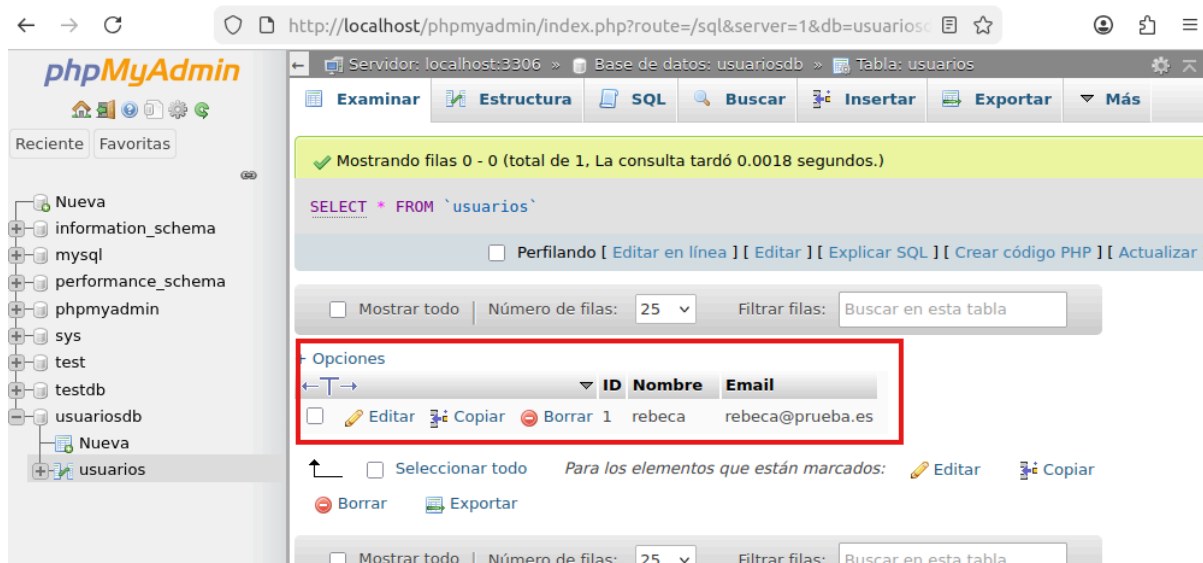
Nota: utilizamos el usuario root para incluir los valores en la tabla

4. Verifica el resultado

Inserta al menos un registro desde el formulario y comprueba en phpMyAdmin que el registro se ha almacenado correctamente en la base de datos.



Datos guardados correctamente



Mostrando filas 0 - 0 (total de 1, La consulta tardó 0.0018 segundos.)

```
SELECT * FROM `usuarios`
```

Perfilando [Editar en línea] [Editar] [Explicar SQL] [Crear código PHP] [Actualizar]

Mostrar todo | Número de filas: 25 | Filtrar filas: Buscar en esta tabla

Opciones

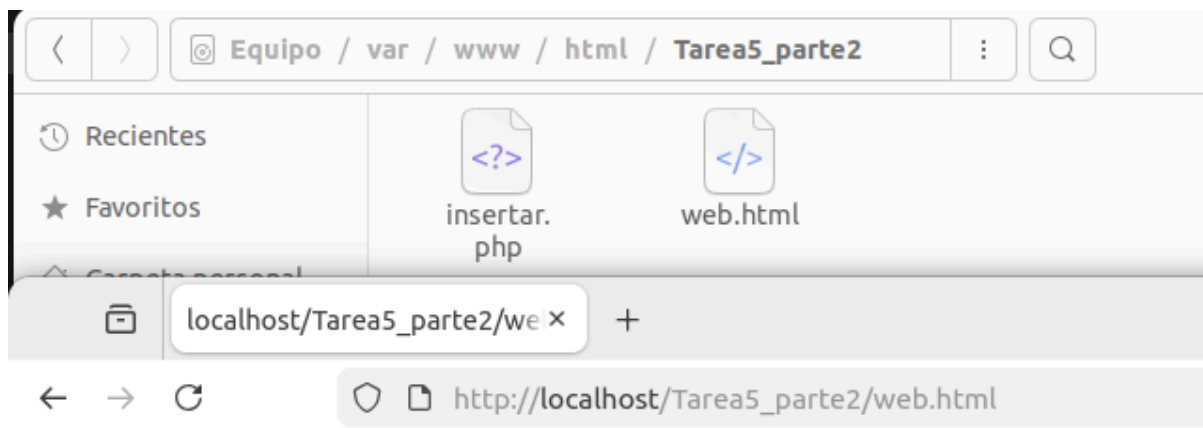
	ID	Nombre	Email
☐ [Editar] [Copiar] [Borrar]	1	rebeca	rebeca@prueba.es

Seleccionar todo | Para los elementos que están marcados: [Editar] [Copiar]

Borrar [Exportar]

Mostrar todo | Número de filas: 25 | Filtrar filas: Buscar en esta tabla

Nota: las pruebas realizarlas desde la dirección <http://localhost/...> (no desde Live Server).
Verifica correctamente tu ruta dentro de /var/www/html/



Equipo / var / www / html / Tarea5_parte2

Recientes

Favoritos

insertar.php

web.html

localhost/Tarea5_parte2/we x

http://localhost/Tarea5_parte2/web.html

Registro de usuarios

Nombre:

Email:

Parte 3 - Peticiones síncronas y asíncronas

Entregable

- Documento (*Parte3.pdf*) **con explicaciones** y las siguientes captura:
 - Funcionamiento de comunicación síncrona
 - Funcionamiento de comunicación asíncrona

Desarrollo

En esta parte vamos a comparar dos formas de comunicación entre el cliente y el servidor:

- **Comunicación síncrona:** se recarga toda la página para obtener la respuesta.
- **Comunicación asíncrona:** se actualiza solo una parte de la página, sin recargarla.

El objetivo es entender la diferencia de funcionamiento entre ambos métodos a través de dos mini-ejercicios.. Revisa la presentación incluida en esta UT.

En la **comunicación síncrona** la solicitud de datos es el siguiente:

HTML (con GET) -> servidor (PHP) -> recarga página

En la **comunicación asíncrona** la solicitud de datos es el siguiente:

Javascript (fetch())-> servidor (PHP)-> actualiza parte de la página

Deberás de crear una base de datos (**nombresdb**) en phpMyAdmin que guarde los siguientes campos en una tabla (**nombres**):

- id (clave primaria, incremento automático)
- nombre

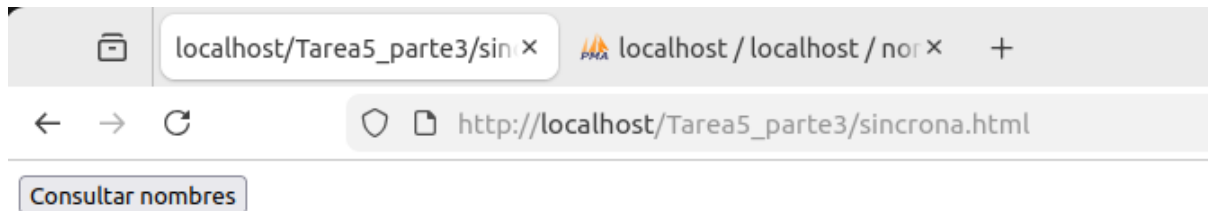
Inserta al menos 3 nombres manualmente desde phpMyAdmin.

☐	Mostrar todo	Número de filas:	25	▼
+ Opciones				
← T →				
			ID	nombre
☐	 Editar	 Copiar	 Borrar	1 Rebeca
☐	 Editar	 Copiar	 Borrar	2 Maria
☐	 Editar	 Copiar	 Borrar	3 Albert

1. Comunicación síncrona

Crea un archivo **sincrona.html** que incluya un botón "Consultar nombres".

```
<form action="sincrona.php" method="get">
  <button type="submit">Consultar nombres</button>
</form>
```

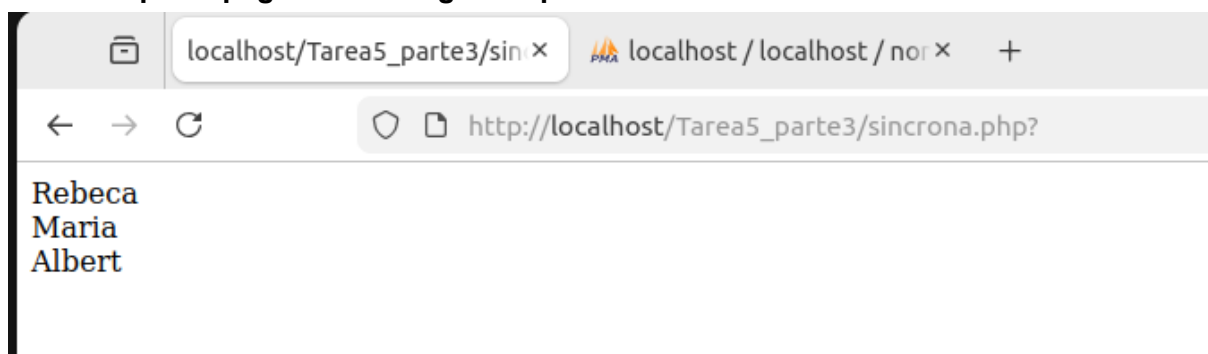


El botón debe enviar una petición GET al archivo **sincrona.php**:

```
sincrona.php
1  <?php
2  // Conectamos con la base de datos
3  $conexion = new mysqli("localhost","root","admin123","nombresdb");
4
5  // Insertamos los datos en la tabla usuarios
6  $resultado= $conexion->query("SELECT nombre FROM nombres");
7
8  // Mostramos los resultados. Fetch_assoc() convierte cada fila en un array
9  while($fila=$resultado->fetch_assoc()){
10     echo $fila['nombre']."<br>";
11 }
12 ?>
```

Nota: fetch_assoc() pertenece a PHP, no a JavaScript. No tiene nada que ver con comunicación asíncrona. Únicamente se usa para recorrer el resultado de una consulta.

Observa que la página se recarga completamente tras enviar el formulario.



2. Comunicación asíncrona

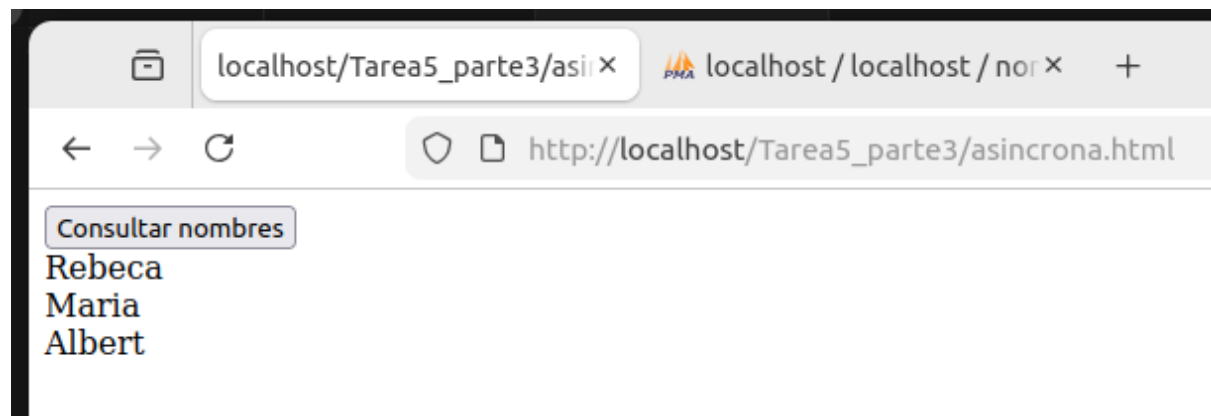
Crea un archivo **asincrona.html** con:

- Botón “Consultar nombres”
- Contenedor vacío para mostrar los resultados (por ejemplo: <div id="resultado"></div>)

Al pulsar el botón, se debe ejecutar un script JavaScript que utilice fetch() para solicitar los nombres al servidor (archivo **sincrona.php**). El script se puede incluir directamente en el archivo html:

```
asincrona.html | sincrona.php | asincrona.html x
asincrona.html > html
1  <!DOCTYPE html>
2  <html lang="es">
3    <header>
4      <meta charset="UTF-8">
5    </header>
6    <body>
7      <button id="boton" type="button">Consultar nombres</button>
8      <div id="resultado"></div>
9
10     <script>
11       const salida=document.getElementById("resultado");
12       const boton=document.getElementById("boton");
13
14       document.getElementById("boton").addEventListener("click", function(){
15         fetch("sincrona.php").then(res=>res.text()).then(datos => { salida.innerHTML=datos;})
16       })
17     </script>
18   </body>
19 </html>
```

En este caso la página no se recarga, **solo se actualiza el contenido del <div> gracias a la comunicación asíncrona.**



Importante! Como se puede observar, la diferencia entre síncrono y asíncrono no está en PHP sino en el navegador.

Criterios de puntuación (10 puntos)

- ❖ Parte 1 - Documento capturas de instalación (3 puntos)
- ❖ Parte 2 - Primer cliente-servidor (3 puntos)
- ❖ Parte 3 - Peticiones síncronas y asíncronas(4 puntos)